



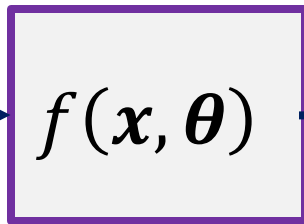
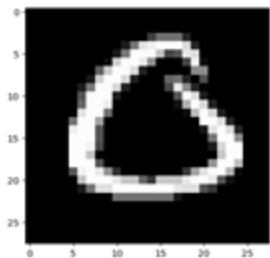
Multilayer Perceptron (MLP)

Rowel Atienza, PhD
University of the Philippines
github.com/roatienza
Updated: 2025

Problem Definition (Supervised Learning)

Given a dataset $\mathcal{D} = (\mathbf{x}, \mathbf{y})$, find a function or model

$$f(\mathbf{x}, \boldsymbol{\theta}): \mathbf{x} \in \mathbb{R}^N \rightarrow \mathbf{y} \in \mathbb{R}^M$$



$$\mathbf{y} \in \mathbb{R}^{10}$$

$$\mathbf{x} \in \mathbb{R}^{28 \times 28 \times 1}$$

What is $f(\cdot)$?

$f(\cdot)$ is generally a non-linear function that maps an input distribution $\mathbf{x} \sim p(\mathbf{x})$ to an output distribution $\mathbf{y} = p(\mathbf{y}|\mathbf{x})$:

$$\mathbf{y} = f(\mathbf{x}) = p(\mathbf{y}|\mathbf{x})$$

$f(\cdot)$ is an estimator of density $p(\mathbf{y}|\mathbf{x})$

What is $f(\cdot)$?

In deep learning, $f(\cdot)$ is parameterized:

$$\mathbf{y} = f(\mathbf{x}; \boldsymbol{\theta}) = p(\mathbf{y}|\mathbf{x}; \boldsymbol{\theta})$$



Neural network parameters

General Function Approximator

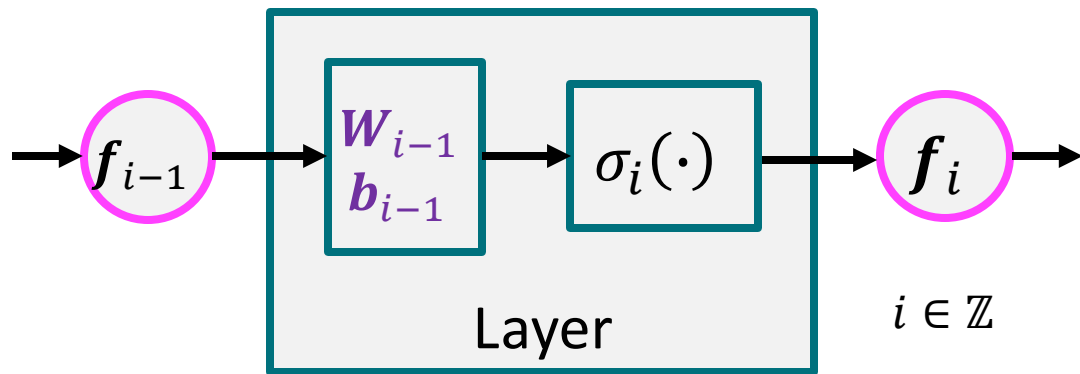
Theorem: Any function $f(\cdot)$ can be approximated by a composition of several smaller functions f_i :

$$\mathbf{y} = f(\mathbf{x}) \approx f_n \circ f_{n-1} \circ f_{n-2} \circ \cdots \circ f_1 (\mathbf{x})$$

$$\ni f_0 = \mathbf{x}, n \in \mathbb{Z}$$

f_i : Keras Dense Layer (Dense) or PyTorch Linear (Linear) + Activation

$$f_i(f_{i-1}; \theta_{i-1}) = \sigma_i(\mathbf{W}_{i-1}f_{i-1} + \mathbf{b}_{i-1})$$

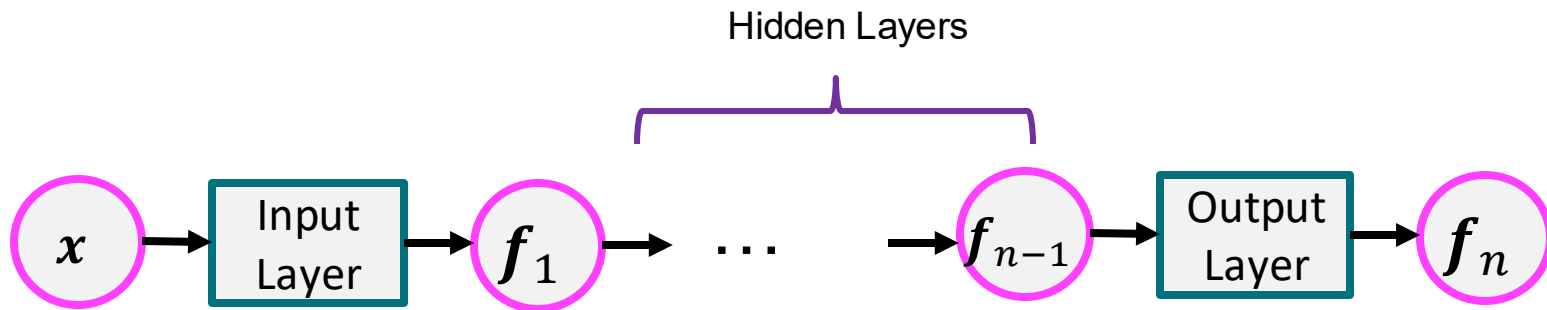


Weights: $\mathbf{W} = \{\mathbf{W}_0, \mathbf{W}_1, \dots, \mathbf{W}_{n-1}\}$ Biases: $\mathbf{b} = \{\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{n-1}\}$

Weights, Biases := Parameters: $\theta = \{\theta_0, \theta_1, \dots, \theta_{n-1}\}$ $\theta_{i-1} = \{\mathbf{W}_{i-1}, \mathbf{b}_{i-1}\}$

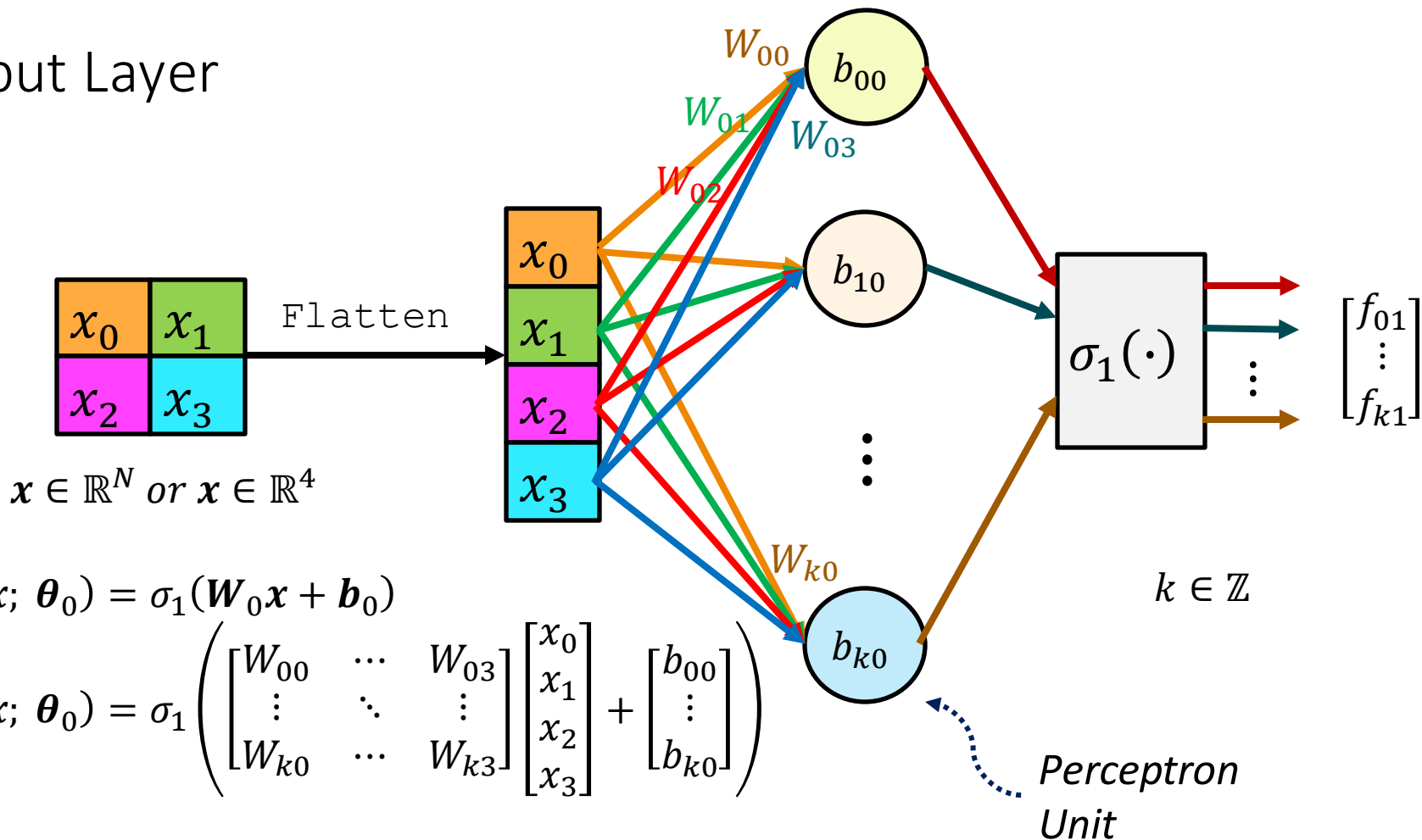
Activation function: $\sigma(\cdot)$

MLP: Function Approximator Implementation

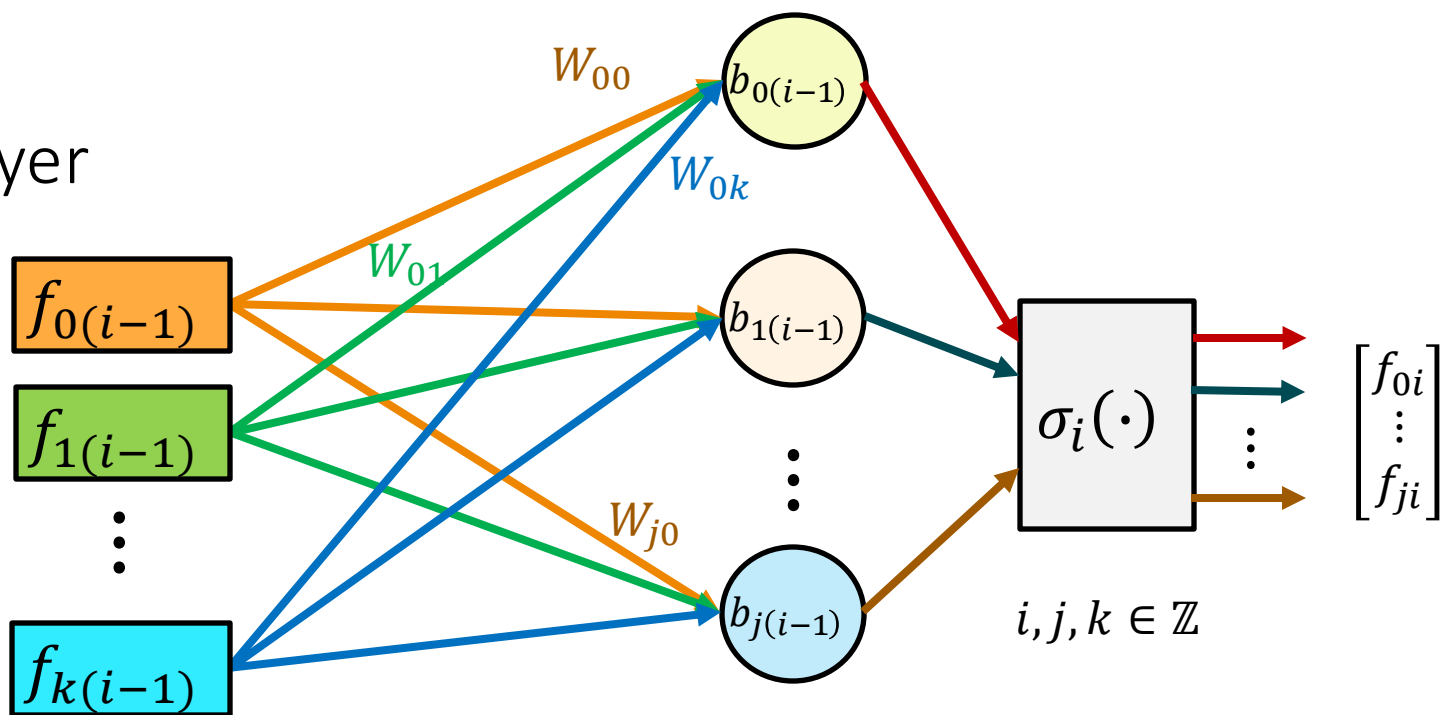


$$\mathbf{y} = f(\mathbf{x}) \approx f_n \circ f_{n-1} \circ f_{n-2} \circ \dots \circ f_1 (\mathbf{x})$$
$$\ni f_0 = \mathbf{x}, n \in \mathbb{Z}$$

Input Layer

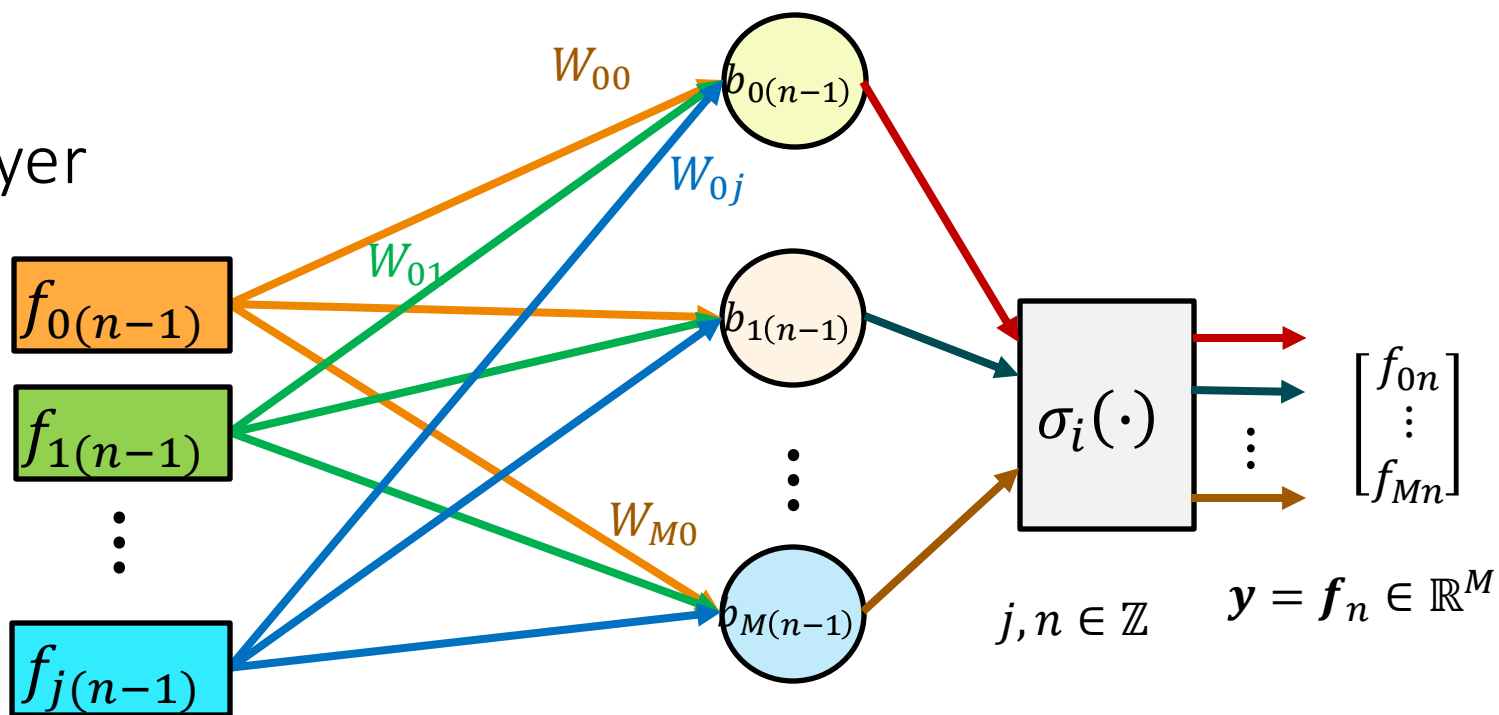


Hidden Layer



$$\mathbf{f}_i(\mathbf{f}_{i-1}; \boldsymbol{\theta}_{i-1}) = \sigma_i \left(\begin{bmatrix} W_{00} & \cdots & W_{0k} \\ \vdots & \ddots & \vdots \\ W_{j0} & \cdots & W_{jk} \end{bmatrix} \begin{bmatrix} f_{0(i-1)} \\ f_{1(i-1)} \\ \vdots \\ f_{k(i-1)} \end{bmatrix} + \begin{bmatrix} b_{0(i-1)} \\ \vdots \\ b_{j(i-1)} \end{bmatrix} \right)$$

Output Layer

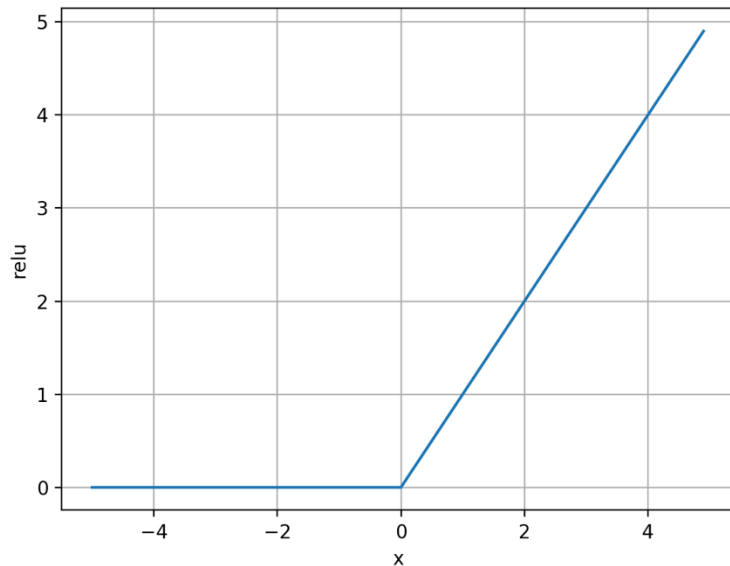


$$\mathbf{f}_n(\mathbf{f}_{n-1}; \boldsymbol{\theta}_{n-1}) = \sigma_n \left(\begin{bmatrix} W_{00} & \cdots & W_{0j} \\ \vdots & \ddots & \vdots \\ W_{M0} & \cdots & W_{Mj} \end{bmatrix} \begin{bmatrix} f_{0(n-1)} \\ f_{1(n-1)} \\ \vdots \\ f_{j(n-1)} \end{bmatrix} + \begin{bmatrix} b_{0(n-1)} \\ \vdots \\ b_{M(n-1)} \end{bmatrix} \right)$$

Activation Function: $\sigma_i(\cdot)$

Rectified Linear Unit:

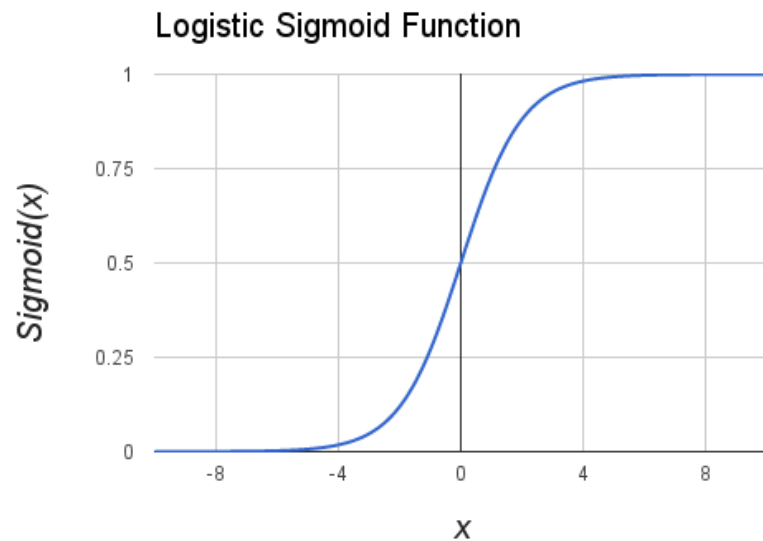
$$\sigma(x) = \text{ReLU}(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$



Activation Function: $\sigma_i(\cdot)$

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



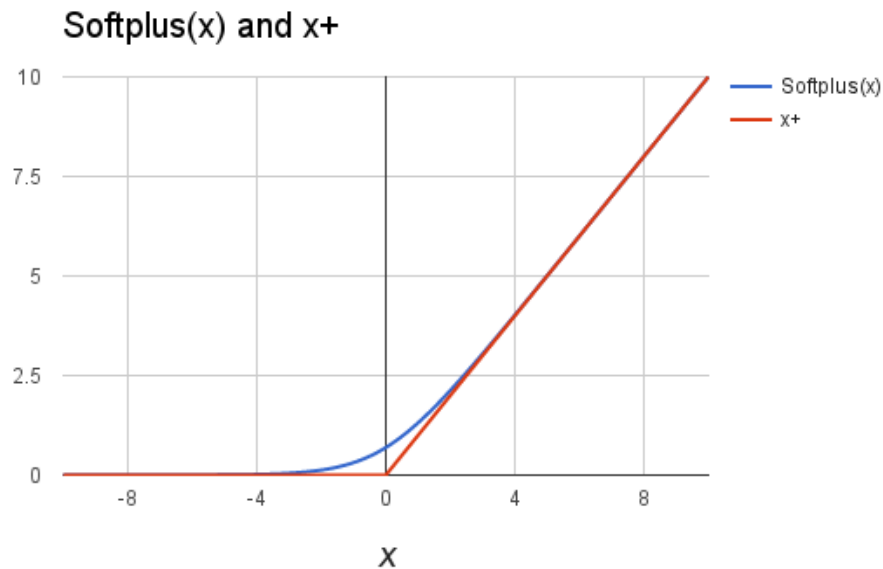
Activation Function: $\sigma_i(\cdot)$

Hyperbolic tangent:

$$\sigma(x) = \tanh x$$

Softplus:

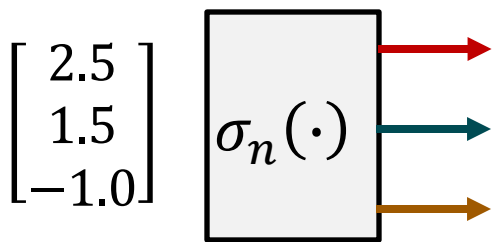
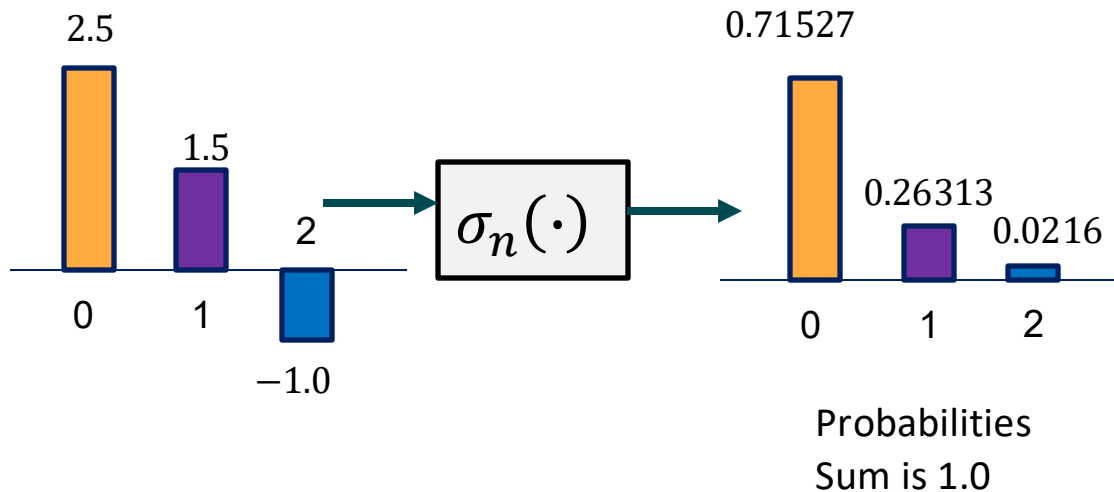
$$\sigma(x) = \ln(1 + e^x)$$



Activation Function: $\sigma_i(\cdot)$

Softmax:

$$\sigma(x_i) = \frac{e^{x_i}}{\sum_{k=0}^C e^{x_k}}$$



$$\sigma_n \left(\begin{bmatrix} f_{0n} \\ f_{1n} \\ f_{3n} \end{bmatrix} \right) = \text{softmax} \left(\begin{bmatrix} 2.5 \\ 1.5 \\ -1.0 \end{bmatrix} \right) = \begin{bmatrix} 0.71527 \\ 0.26313 \\ 0.02160 \end{bmatrix}$$

Which activation to use?

Input and Hidden Layers

ReLU, GELU – inject non-linearity

Softplus – used in deep reinforcement learning

Linear – pass through

Output Layer

Sigmoid – Bernoulli Distribution, Normalized Linear Regression

Softmax – Logistic Regression

Linear – Un-normalized Linear Regression

How to learn $f(\cdot)$ from data?

Recall: Norms, Metrics,
Distances from ML
Objective is to reduce the
distance of the *prediction* $\mathbf{y} = f(\mathbf{x})$ from the ground truth
label $\tilde{\mathbf{y}}$

This distance, norm, or metric
is oftentimes called a **Loss
Function** or an **Objective
Function**

Loss Function	Equation
Mean Squared Error (MSE)	$\sum_{i=1}^{categories} (y_i^{label} - y_i^{prediction})^2$
Mean Absolute Error (MAE)	$\sum_{i=1}^{categories} y_i^{label} - y_i^{prediction} $
Categorical Cross Entropy (CE)	$- \sum_{i=1}^{categories} y_i^{label} \log y_i^{prediction}$
Binary Cross Entropy (BCE)	$-y_1^{label} \log y_1^{prediction} - (1 - y_1^{label}) \log(1 - y_1^{prediction})$

Optimization

Given the dataset $\{\mathcal{D}_{train}, \mathcal{D}_{test}\} = \{(\mathbf{x}_n, \mathbf{y}_n), (\mathbf{x}_m, \mathbf{y}_m)\}$, we minimize the loss function on $\mathcal{D}_{train}$ and we measure the performance on $\mathcal{D}_{test}$

Optimization Algorithm: Stochastic Gradient Descent (SGD)

Variants of SGD: Adam, AdamW

Optimization Recipe

Initialize all weights by random values

Better initializers: Kaiming, Glorot, Uniform, Normal, LeCun,

Biases by zero or small positive values

Usually, default initialization algorithms are good enough

Preprocessing of Data

Input

Normalize such that $x_i \in [0., 1.]$

Adjust such that inputs has zero mean and unit variance

Output

In logistic regression, convert all labels to one-vectors

Example: In MNIST, digit 8 label is $\tilde{y} = [0,0,0,0,0,0,0,1,0,0]^T$

In linear regression, normalize outputs such that $y_i \in [0., 1.]$ or such that $y_i \in [-1., 1.]$

Hyper-parameters

Tunable network parameters

Depth or value of n in f_n

Width values of k and j in
the input and hidden layers

Tunable training parameters

Learning rate

Learning rate scheduler

Warm-up

Batch size, Epochs

Optimization algorithm

In Summary

MLP is an implementation of the general function approximator

MLP is made of layers as building blocks

Design choices such as hyper-parameters, activation functions, etc

END